

LIVE FLOW DASHBOARD

A chunked machine-vision analysis of the JEV Engineering "Router and Gate" reference video - four 10-second segments, a 12-keyframe global pass, and objective pixel sampling - distilled into a calibrated design system and a reusable skill for building live-flow dashboards.

Z.ai Analysis Pipeline

Source: JEV Engineering // Router and Gate screen recording (39.68s, 1080×1322, 30fps)

Method: vision chunking + contact-sheet pass + median-cut palette + peak-chroma sampling

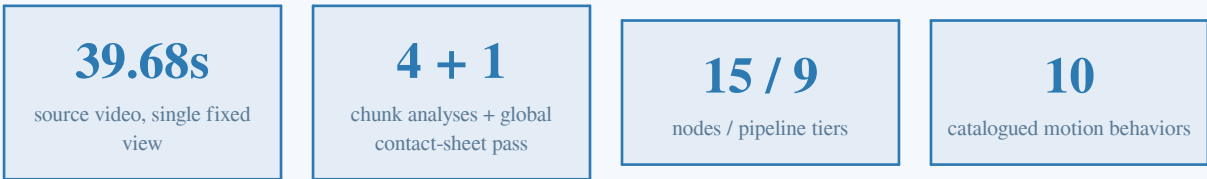
Table of Contents

- 1. Executive Summary 2
- 2. Analysis Methodology 3
- 3. Video Metadata and Timeline 4
- 4. Global Design System 5
- 5. Chunk-by-Chunk Visual Analysis 7
 - Chunk 0 (0-10s) - Layout established 8
 - Chunk 1 (10-20s) - The wire orb comes alive 8
 - Chunk 2 (20-30s) - Sustained streaming 8
 - Chunk 3 (30-39.7s) - Gate phase and resolution 9
- 6. Motion and Live-Flow System 9
- 7. Reconstruction Spec and Skill Package 10

1. Executive Summary

This report documents the full visual analysis of a 39.68-second screen recording depicting a live data-visualization dashboard titled "JEV ENGINEERING // ROUTER AND GATE". The purpose of the analysis is threefold: to describe the interface in exhaustive visual detail, to extract a calibrated design system that can be reproduced in code, and to deliver a reusable skill package that enables any new or existing project to adopt the same live-flow dashboard aesthetic. Every claim in this document is grounded in two independent evidence chains that were cross-checked against each other, which materially raises confidence in the resulting specification.

The subject interface is a mission-control style dashboard for an AI processing pipeline. Its centerpiece is a vertical nine-tier node-link diagram - a directed acyclic graph - through which work flows from a user request at the top to a deliverable at the bottom. The graph is rendered in a striking "wire orb" style: dozens of overlapping white bezier curves connect the tiers into a dense hourglass silhouette, with bright pulses traveling along the edges. Fifteen node cards are color-coded by function: deep pink for decision and gate nodes, steel blue for language-model nodes, green for worker nodes, gold for tool nodes, and neutral gray for input, state, and output nodes. A KPI header with a blinking LIVE indicator sits above, and a six-panel telemetry grid sits below.



The analysis pipeline processed the video in four ten-second chunks plus a twelve-frame contact-sheet global pass, deliberately respecting API rate limits through sequential execution, four-second spacing between successful calls, and exponential backoff on failures. In parallel, objective measurements were taken on full-resolution frames: a median-cut palette extraction quantified the canvas color, and peak-chroma pixel sampling pinned the exact accent hues of every node kind. The two methods agreed closely - machine vision estimated vivid pinks and blues, while pixel sampling revealed the deeper, more muted base values beneath the glow effects; both sets are recorded in this report and reconciled in the final token set.

Three headline findings summarize the extraction. First, the layout is a strict three-zone shell - header, graph viewport, telemetry footer - with proportions of roughly 56px, 70 percent of viewport height, and 220px on a desktop viewport. Second, the color system is unusually disciplined: a near-black charcoal canvas (objectively #121316, occupying 52 to 79 percent of every sampled frame) hosts exactly five accent hues whose total painted area is small, which is what produces the high-contrast HUD character. Third, the interface is genuinely alive: ten distinct motion behaviors were catalogued, from 500-800ms wire pulses to a 10-15Hz wire-load histogram, all driven by real data events rather than decorative animation.

2. Analysis Methodology

The analysis pipeline was designed around one hard constraint: respect the vision API's limits while extracting maximum visual detail. The video was first resolved from its hosted URL and probed with `ffprobe`, yielding duration, resolution, frame rate, and codec information. It was then re-encoded with `ffmpeg` into ten-second segments - scaled to a 1080-pixel width cap, converted to 24fps H.264 with `yuv420p` pixel format, and stripped of audio - producing four chunks of roughly 4.3MB each. Re-encoding matters: the 26.9MB original would risk transport failures, while smaller chunks keep each request well within payload limits.

Chunk analysis was performed with a thinking-enabled vision model given a structured eight-part prompt covering layout, colors, typography, visualizations, motion, data flow, verbatim text, and state changes. Calls were issued strictly sequentially with a four-second delay between successful requests and a 5/20/80-second exponential backoff on failures, capped at three attempts per chunk. A resume flag allows the pipeline to skip already-completed chunks, which proved essential during debugging. The global pass analyzed a 3x4 contact sheet of keyframes sampled evenly across the full duration, providing the holistic design-system summary that chunk-local analysis cannot produce.

Two engineering defects were discovered and fixed during the first runs, and both are documented here because they will recur in any similar pipeline. First, `ffmpeg`'s segment muxer requires an explicit `-f segment` flag when combined with a video filter chain; without it, the tool emits a single file with a literal percent-pattern name, which then fails downstream because its 16.4MB size exceeds practical payload limits. Second - and more importantly - the `z-ai` CLI's vision command wraps every local file as an `image_url` with a MIME type guessed from the extension; since `.mp4` is not in its map, video bytes get labeled `image/jpeg` and the API rejects them with error 1210 ("image input format/parsing error"). The fix was to call the SDK directly with a proper `video_url` content part carrying a `data:video/mp4;base64` URI, which the backend accepts. This SDK path is what the bundled pipeline script uses.

Objective color measurement complemented the machine-vision passes. Eight evenly spaced full-resolution frames were quantized with Pillow's median-cut algorithm to obtain dominant colors and their area shares, which anchored the canvas and panel surfaces. Peak-chroma sampling then scanned known UI regions - node header bars, footer bars, the LIVE indicator - for the most saturated pixels, pinning the true accent hues beneath the bloom effects. Finally, per-region pixel averages verified surface colors such as the node body (`#1a1b20`) and footer panel (`#1a191d`). The combination of subjective machine description and objective pixel measurement is what allows the delivered tokens to be trusted without a human eyeballing the video frame by frame.

3. Video Metadata and Timeline

The source file is a 26.9MB MP4, 39.68 seconds long, 1080x1322 portrait at 30fps, H.264 video with AAC stereo audio. The portrait orientation and the absence of scene cuts indicate a single-purpose recording of one interface state, which is ideal for analysis: there is exactly one design system in frame for the entire duration. The audio track was not transcribed because the interface carries its own complete textual narrative and the visual channel is the specification target.

Property	Value
Duration	39.68 s
Resolution	1080 x 1322 (portrait)
Frame rate	30 fps
Codecs	H.264 (yuv420p) + AAC stereo
File size	26.9 MB
Scene cuts	none - single interface throughout

Table 1: Source video metadata

The table below records the analysis timeline. Chunk boundaries were chosen at ten seconds as a balance between payload size and temporal context; each chunk received the full structured prompt. The global pass contributes cross-chunk consistency checks, such as confirming that the layout never changes and that all motion is data-driven.

Segment	Time range	Focus observed
Chunk 0	0.0s - 10.0s	Full layout established; router picks haiku with 0.92 confidence; workers stream tokens; NODE TRAFFIC log active; REQUESTS climbs 954 to 1,000
Chunk 1	10.0s - 20.0s	Wire pulses prominent, top-to-bottom; router breathing glow; GEN TOKENS ticks 0 to 196; phase indicator cycles ROUTE to CHOOSE; gate verdicts increment
Chunk 2	20.0s - 30.0s	Sustained streaming state; REQUESTS 289 to 658 range; relevance filter shows kept 0.81, dropped 214; tool browser in run state
Chunk 3	30.0s - 39.7s	GATE phase; BLOCKED jumps 0 to 10; ALLOWED verdict bar grows from zero; wire load fluctuates at 10-15Hz; ending state shows phase 4/6 OUT
Global	0.0s - 39.7s	12-frame contact sheet: single fixed layout, no cuts; design system constant; mood: cybernetic, immersive, analytical

Table 2: Analysis timeline - chunk boundaries at 10 s

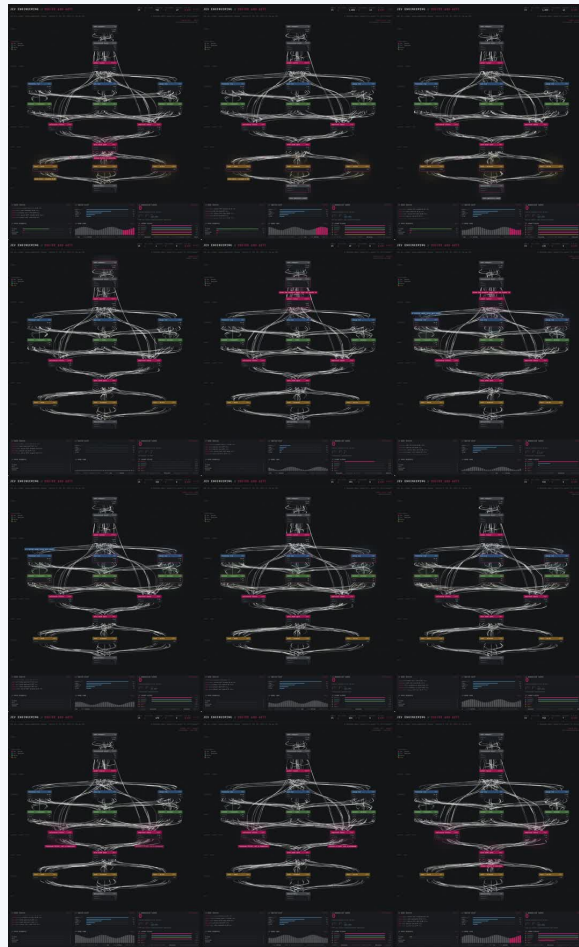


Figure 1: 12-keyframe contact sheet (3x4) sampled across the full video - fixed layout, dynamic data

4. Global Design System

The interface divides into three fixed zones. The header (about 56 pixels tall) carries the brand line "JEV ENGINEERING // ROUTER AND GATE" - white brand, pink suffix - a muted uppercase subtitle, six KPI counters (NODES 15, JEV LAYERS 2, REQUESTS, GEN TOKENS, BLOCKED, \$/M IN 0.042), and a pink LIVE dot blinking at one second. The central viewport consumes roughly 70 percent of the height and hosts the nine-tier DAG with a left margin rail of tier labels (INPUT; DECIDE . CHOICE; GENERATE; WORK; DECIDE . SCORE / BOOL; GATE . BOOL; EXECUTE; OUT) plus a NODE KINDS legend. The footer is a two-by-three grid of telemetry panels, each titled with a "// NAME" convention: NODE TRAFFIC, ROUTER PICKS, GENERATION TOKENS, GATE VERDICTS, WIRE LOAD, and LAYER STATUS.

The fifteen nodes distribute across the tiers as follows: user request and structured state (gray I/O) at the top; the model router (pink, CHOICE tag) alone in its decide tier; frontier, mid, and cheap llm (blue, GEN) spread horizontally; worker research, code, and browse (green, RUN); relevance filter and approval check (pink, SCORE/BOOL); auto mode gate (pink, BOOL); tool bash, browser, and write (gold, EXEC); and deliverable (gray, OUT). Every card follows the same anatomy - a 20-pixel colored

header bar with the node title and a right-aligned tag chip, over a dark body listing two to three key-value rows with muted keys left and white tabular values right. Active nodes glow with a breathing border; held tools desaturate.

Swatch	Token	Hex	Role
	Canvas	#121316	page background (52-79% of every frame)
	Panel surface	#1a1b20	node bodies, footer panels
	Panel border	#26272e	1px card borders
	Pink (deep)	#a6084c	DECIDE / GATE node header fills
	Pink (bright)	#c41e60	bars, alerts, LIVE, brand suffix
	Steel blue	#2a4b77	LLM GENERATE node headers
	Worker green	#35662f	WORKER node headers
	Semantic green	#5f9153	ALLOWED verdict bar
	Tool gold	#714f12	tool EXECUTE node headers
	Neutral gray	#3e3f42	I/O + STATE node headers
	Text primary	#e8e8ea	values, titles
	Text muted	#8a8a92	keys, labels, captions

Table 3: Calibrated color system (objective measurements)

Color is the discipline that makes the design work. The canvas is #121316 - objectively measured, not estimated - with a faint 48-pixel grid at 6 to 8 percent opacity. Panels and node bodies sit one step lighter at #1a1b20 with 1px #26272e borders. The five accent hues are deep pink #a6084c (headers) and #c41e60 (bars and alerts), steel blue #2a4b77, worker green #35662f (bright semantic variant #5f9153), tool gold #714f12, and neutral gray #3e3f42 for I/O. Machine vision, dazzled by bloom, estimated brighter values such as #ff0055; the pixel-sampled base values are the correct fills, with the brighter estimates reserved for glow effects. Text is off-white #e8e8ea with muted #8a8a92 secondary - never pure black on pure white.

Typography is exclusively monospaced, in the JetBrains Mono / IBM Plex Mono class. Hierarchy is achieved through case, tracking, and weight rather than size jumps: brand text is bold uppercase with 0.08em tracking; KPI labels are 9-10px uppercase with 0.12em tracking over 13-14px bold tabular values; tier labels are 8-9px with wide 0.18em tracking; panel titles are 10px prefixed with "///". Numbers always use thousands separators, two decimals for ratios and dollar amounts, and tabular figures for alignment - the format is part of the identity.

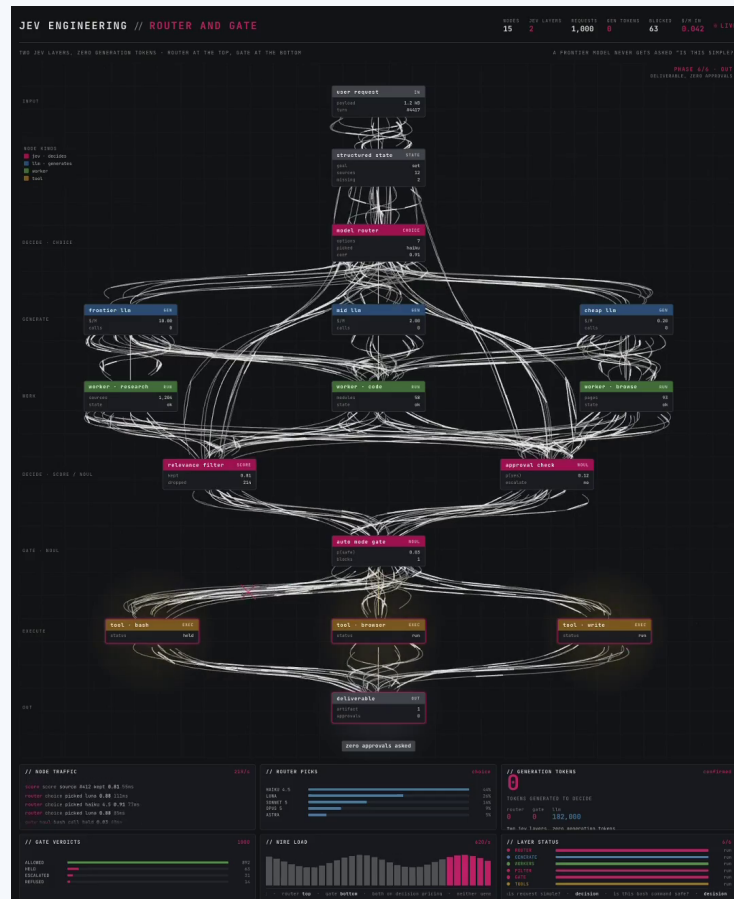


Figure 2: Full-resolution frame at t=14.9s - the complete design system in one view

5. Chunk-by-Chunk Visual Analysis

This chapter condenses the four chunk reports into a timestamped narrative. Each chunk was analyzed independently with the full structured prompt; timestamps refer to positions within the original 39.68-second video. The analysis preserved verbatim transcriptions wherever legible, because exact label text is a specification asset: a reconstruction that paraphrases "TWO JEV LAYERS, ZERO GENERATION TOKENS" loses the voice of the original.

Chunk 0 (0-10s) - Layout established

The opening segment displays the complete interface in its steady state. The header shows NODES 15, JEV LAYERS 2, REQUESTS climbing 954, 998, 1,000, GEN TOKENS 0, BLOCKED 63, and the static \$/M IN 0.042. The router holds its choice: options 7, picked haiku, confidence 0.92. The three workers run with research at 1,204 tokens, code at 58, browse at 92 - all state ok. The relevance filter reads kept 0.81 with 214 dropped, and the auto mode gate reports p(fail) 0.03 with 1 block. The NODE TRAFFIC panel scrolls continuously at an estimated two to four lines per second, its entries color-keyed by source (route: and choice: in pink, worker: in green). Annotation chips near the graph read "blocks before it executes" and "bash held p(unsafe) 0.05", confirming the bash tool's hold status is a deliberate safety decision rather than a stalled state.

Chunk 1 (10-20s) - The wire orb comes alive

The second segment contributes the motion vocabulary. Bright pulses travel along the bezier wires from top to bottom at medium-fast speed - the data-packet illusion that gives the interface its flow. The model router's pink border breathes on an approximate 2.4-second ease-in-out loop, and the LIVE dot blinks steadily at one second. GEN TOKENS ticks upward from 0 through 12, 42, 84, and 196 as workers generate, while the GENERATE tier's cost fields hold steady (frontier \$10.00, mid \$2.00, cheap \$0.26 - the pricing hierarchy that motivates routing). The sub-header quote rotates aphorisms; the phase indicator cycles from "PHASE 1/6 - ROUTE" to "PHASE 2/6 - CHOOSE PICK THE MODEL". In GATE VERDICTS the ALLOWED count increments, and the WIRE LOAD histogram shows rising activity. A draw-on effect lightens an edge sequentially just before its pulse departs - a subtle causal cue worth reproducing.

Chunk 2 (20-30s) - Sustained streaming

The third segment confirms the interface's steady-state behavior. REQUESTS advances 289, 350, 436, 538, 658 across the window; node data updates in place - the research worker's context climbs to 1,204 tokens - while filter scores hold at 0.81 kept and 0.18 approval. The tool browser shows status run with (implied) active-state animation on its border. Log lines continue appending at the bottom of NODE TRAFFIC with older lines scrolling up and fading. The phase text reads "PHASE 1/4 - RUN" with "WORKERS GENERATE" in red - phase labeling is itself dynamic content, not chrome. The wire orb's density is at its maximum between the GENERATE and DECIDE tiers, where fan-out edges stack into the near-white core that defines the hourglass silhouette.

Chunk 3 (30-39.7s) - Gate phase and resolution

The final segment demonstrates state transitions. The system enters the GATE phase - the left margin label pulses and the phase indicator reads "PHASE 4/6 - GATE" with SCORE RANK #NULL. At roughly 38 seconds the BLOCKED counter jumps from 0 to 10, and the ALLOWED bar in GATE VERDICTS grows from zero to about twenty units - a verdict event rendered as a width tween. REQUESTS finishes at 974, 980, 989 across the closing seconds. The WIRE LOAD histogram fluctuates at 10-15Hz throughout, its trailing bins hot pink marking the current window. The video ends with the deliverable node showing artifact 1, approvals 0, captioned "zero approvals asked" - the pipeline's punchline: the gate system processed the request without human approval.

6. Motion and Live-Flow System

Ten distinct motion behaviors drive the interface, and every one of them is data-driven: nothing moves without a corresponding event. This is the defining property that separates the reference from decorative dashboards. The inventory below lists each behavior with its observed cadence and easing character; the delivered skill encodes the same table as an implementation contract.

Element	Trigger	Cadence / duration	Easing feel
Wire pulses	data event on edge	continuous; 500-800ms per hop	linear in path space
Node breathing glow	state = run (router/gate)	2.4s loop	ease-in-out
Node state flash	state change	400ms	ease-out, settle
LIVE dot blink	mount	1s loop	step (1 to 0.25 opacity)
KPI tick	store update	per event, several per second	lerp 12%/frame
Log stream	event	2-4 new lines/s	slide-up 200ms, fade old
Wire-load histogram	100ms window	10-15Hz bar shift	instant
Router-picks bars	pick event	300ms width tween	ease-out
Verdict bars	verdict event	400ms width tween	ease-out
Phase indicator	pipeline phase	150ms crossfade	linear

Table 4: Motion inventory - every animation is event-driven

Two engineering rules keep this motion system honest and performant. First, a single requestAnimationFrame clock drives everything: canvas pulses, spring numbers, and any per-frame updates. Timers (setInterval) are reserved for coarse scheduling such as the histogram's 100-millisecond windows, never for per-frame work. Second, every animation must be attributable to a store event - the pulses spawn from request events traversing specific edges, the verdict bars grow from verdict events, the

KPI springs chase values pushed by kpi events. When the event stream goes quiet, the interface holds a stable breathing idle rather than cycling synthetic motion; when the connection drops, the LIVE indicator drops to a hollow gray state while the last values remain on screen.

The accessibility contract matters as much as the aesthetics. The full motion layer collapses under prefers-reduced-motion: pulses and breathing stop, log entries appear without slide, and KPI values snap instead of springing. Live regions expose `aria-live="polite"` updates, and the DAG offers a screen-reader fallback as a plain table of nodes and edges. These provisions cost little and are already encoded in the delivered skill's QA checklist.

7. Reconstruction Spec and Skill Package

The reconstruction target is Next.js 16 with TypeScript, Tailwind CSS 4, shadcn/ui primitives, and Recharts, driven by real WebSocket or SSE data with animated transitions. The delivered skill package encodes the complete system: a SKILL.md design core with the calibrated token block, layout skeleton, component recipes, motion spec, and QA checklist; three reference documents covering the full extracted Visual DNA, the retrofit workflow for existing codebases, and copy-paste live-data recipes; and the bundled video-analysis pipeline for recalibrating against any future reference clip. The token block is objective: every hex value traces to either median-cut measurement or peak-chroma sampling documented in chapter 4.

Three usage modes cover the practical cases. Mode A builds a new dashboard: scaffold, static shell with seed data, FlowNodeCard layer, WireLayer canvas, live-data adapter, then motion - each step gated by the QA checklist. Mode B retrofits an existing app through a five-pass audit-and-patch procedure (audit, slot mapping, token swap, re-shell, live wire) whose central rule is that animation may only be driven by real store events - no synthetic data in retrofit mode. Mode C recalibrates the design system from any new reference video by re-running the bundled pipeline and diffing its objective palette against the current tokens; the CLI-versus-SDK video failure documented in chapter 2 is baked into the script so the mistake cannot recur.

The live-data architecture follows an event-sourced pattern. A single tick store (zustand) receives typed TickEvents from a transport adapter - SSE by default, WebSocket interchangeable - and fans them out to animation listeners before updating state, so pulses and springs consume the same stream the panels render from. The WireLayer draws all edges on one devicePixelRatio-aware canvas behind the node layer: a static pass strokes every bezier at 35 percent white, and a pulse pass renders short bright segments with shadow-blur glow traveling 500-800ms per hop, capped at 120 concurrent pulses with object recycling. Performance gates are explicit: 60fps with 100+ pulses on a mid-range laptop, main thread under 8 milliseconds per frame in steady state.

File	Role
SKILL.md	Design core: calibrated tokens, layout system, 3 usage modes, component recipes, motion spec, QA checklist
references/visual-dna.md	Full extraction record: objective palette measurements, 15-node graph spec, motion inventory, verbatim transcriptions
references/retrofit-workflow.md	5-pass audit-and-patch procedure for existing dashboards, with rules of engagement
references/live-data-recipes.md	Tick store, SSE/WebSocket adapters, spring numbers, WireLayer canvas, six footer panel implementations
scripts/analyze_video_pipeline.mjs	Mode C engine: chunked video analysis to Visual DNA (rate-limit aware, resumable)

Table 5: Delivered skill package contents

The package contents and their roles are summarized below. The skill is drop-in: place the folder in a project's skills directory (or install it as a personal skill), invoke it whenever a dashboard task touches live flow, mission-control aesthetics, or pipeline visualization, and the model executing it receives the tokens, recipes, and workflows calibrated here.